ARTIFICIAL INTELLIGENCE

&

MACHINE LEARNING LABORATORY

HMLAL701

LABORATORY MANUAL

B. Tech. Computer Engineering

INSTITUTE
Fr. C. Rodrigues Institute of Technology
Vashi, Navi Mumbai400703

DEPARTMENT
Computer Engineering
Honours Level Laboratory | Credits: 2

Course Overview

Course Code: HMLAL701
Course Title: Artificial Intelligence & Machine Learning Laboratory
Credits: 02 | Examination: Continuous Assessment Only (50 Marks)
Pre-requisite: Python Laboratory (CESBL301 / equivalent)

Course Objectives

- Apply data pre-processing, visualization, and supervised learning for structured data.
- Design and train deep neural networks using modern optimization and regularization.
- Implement advanced architectures (CNN, RNN, Autoencoder, GAN) for real-world problems.
- Foster self-learning through global guided projects and certification-based enrichment.

CO-PO Mapping

CO ID	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
HMLAL701.1	3	3	3	3	3	3					
HMLAL701.2	3	3	3	3	3	3	2	3			
HMLAL701.3	3	3	3	3	3	3	2	3			
HMLAL701.4		3	3		3	3	3	3			3
Average	3	3	3	3	3	3	3	3			3

Assessment Structure

Component	Description	Marks
Task Evaluation	Practical tasks 1–8	20
Guided Project	Practical 9 & 10 Coursera / Kaggle	20
Regularity & Participation	Attendance & Engagement	10
TOTAL		**50**

Practical Index

No.	Title	CO | LO	Bloom Level
1	Develop A Machine Learning Pipeline For Healthcare Data	CO1 | LO1.1	Apply
2	Evaluate The Various Classification Model Performance on Fintech Dataset	CO1 | LO1.2	Evaluate
3	Build A Simple Neural Network Using Perceptron and MLP Models	CO2 | LO2.1	Create
4	Create A Deep Neural Network For Image Classification	CO2 | LO2.2	Create
5	Evaluate The Impact Of Regularization Techniques in Deep Neural Networks	CO2 | LO2.3	Evaluate
6	Develop CNN-Based Healthcare Image Classification	CO3 | LO3.1	Create
7	Create RNN And LSTM Model to Perform For Time-Series Forecasting	CO3 | LO3.2	Create
8	Develop An Autoencoder And GAN for Medical Image Processing	CO3 | LO3.3	Create
9	Self Learning – Online Course1 - **Industry-recognized guided project or Coursera capstone** in ML/DL/AI.	CO4 | LO4.1	Create
10	Self Learning – Online Course 2- **Industry-recognized guided project or Coursera capstone** in ML/DL/AI.	CO4 | LO4.1	Create

Industry Perspective Domain Mapping

P#	Industry Domain	Key Technique	Representative Company
1	Hospital Analytics	Regression Pipeline	Practo, Apollo AI
2	Fraud Detection	Classification + ROC-AUC	Razorpay, Paytm
3	Customer Segmentation	Perceptron, MLP	Amazon, Netflix

4	OCR & Autonomous Systems	DNNMNIST / CIFAR-10	India Post, HDFC Bank
5	Reliable AI Systems	Dropout, Weight Decay	Waymo, Google Cloud
6	Medical Diagnostics	CNN + Transfer Learning	Qure.ai, Niramai
7	Algorithmic Trading	RNN, LSTM	Zerodha, Goldman Sachs
8	Medical Image Enhancement	Autoencoder, GAN	GE HealthCare, Siemens
9	Industry Certification	Self-directed Learning	Coursera, Kaggle, fast.ai

PRACTICAL NO. 1

Healthcare ML Pipeline Ethical Data Handling and Regression Modelling

Course Outcome	Learning Outcome	Performance Indicators
CO1: Create and evaluate end-to-end ML solutions with responsible data handling	LO 1.1: Create an ethical, privacy-preserving ML pipeline for healthcare data	PI-1.1.1, 1.4.1, 2.1.2, 2.4.1, 5.1.1, 5.2.2, 6.3.1, 7.1.1, 7.2.1, 8.2.1

1. Aim / Objective

To build an ethical, privacy-preserving machine-learning pipeline using a healthcare dataset; apply data loading, preprocessing (cleaning, imputation, scaling), exploratory visualization, feature engineering, and regression modeling while documenting societal and clinical implications.

2. Rationale

Healthcare is one of the most impactful domains where ML can save lives. Yet it is also one of the most sensitive patient data carries privacy obligations under regulations like HIPAA and India's DPDP Act 2023.

This practical bridges the gap between theoretical ML knowledge from Semester VI and real-world, ethical implementation. Students learn not just to build a pipeline but to ask: Who owns this data? Can our model harm vulnerable patients? How do we document our choices?

Real-world relevance: Hospitals use regression models to predict patient readmission risk, ICU stay duration, and resource allocation all directly impacting patient outcomes and hospital costs.

3. SDG Mapping

SDG No.	SDG Title	Mapping Level	Justification
SDG 3	Good Health and Well-Being	High	Uses healthcare datasets for disease prediction, supporting better clinical decision-making and health outcomes.
SDG 10	Reduced Inequalities	Medium	Ethical, bias-aware modelling ensures healthcare ML tools serve all demographic groups fairly.
SDG 4	Quality Education	High	Focusing on ML pipeline.

4. Prerequisite Knowledge

- Python programming (loops, functions, OOP basics)
- Descriptive statistics: mean, median, standard deviation, correlation
- Supervised learning concepts: regression, train-test split, cross-validation
- Pandas, NumPy, Matplotlib, Seaborn fundamentals
- Awareness of healthcare data privacy (HIPAA, DPDP Act 2023)

5. Short Theory

5.1 The ML Pipeline

A Machine Learning pipeline is a structured sequence of steps that transforms raw data into a deployable model. For healthcare data, this pipeline must incorporate privacy-by-design principles at every stage.

5.2 Key Steps

- Data Collection & Loading: Source data from public repositories (Kaggle, UCI ML Repository). Anonymize PII fields.
- Exploratory Data Analysis (EDA): Understand distributions, identify outliers, visualize class imbalance.
- Data Preprocessing: Handle missing values (mean/median/KNN imputation), remove duplicates, fix data types.
- Feature Engineering: Create new features, encode categorical data, normalize/standardize numeric.
- Model Training: Apply Linear Regression to predict continuous targets (e.g., patient LOS).
- Model Evaluation: Use MAE, MSE, RMSE, R^2 as metrics.

5.3 Linear Regression Formula

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Where $\hat{y}$ is the predicted output, β_0 is the intercept, and $\beta_1 \ldots \beta_n$ are learned coefficients for features $x_1 \ldots x_n$.

5.4 Evaluation Metrics

$MAE = (1/n)\ \Sigma|y_i - \hat{y}_i|$ | $RMSE = \sqrt{[(1/n)\ \Sigma(y_i - \hat{y}_i)^2]}$ | $R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]$

5.5 Privacy-Preserving Practices

- Remove direct identifiers (name, Aadhaar, phone) before analysis.
- Use k-anonymity: ensure each record is indistinguishable from at least k-1 others.
- Document data provenance and usage purpose.

6. Software and Tools Required

- Python 3.10+
- Google Colab (free cloud GPU/CPU)
- scikit-learn 1.4+
- Pandas, NumPy
- Matplotlib, Seaborn
- Kaggle API (for dataset download)

7. Dataset Description

Dataset: Diabetes Health Indicators Dataset (Kaggle / UCI)

Source: https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset

Size: ~70,000 records, 21 features

Target Variable: Diabetes_binary (for classification) or BMI (for regression task)

Key Features: HighBP, HighChol, CholCheck, BMI, Smoker, Stroke, HeartDiseaseorAttack, PhysActivity, Fruits, Vegs, HvyAlcoholConsump, AnyHealthcare, NoDocbcCost, GenHlth, MentHlth, PhysHlth, DiffWalk, Sex, Age, Education, Income

8. Problem Statement

A regional hospital network wants to predict the BMI of patients based on their lifestyle and health indicators to identify patients at risk of diabetes-related complications. Build an ethical ML regression pipeline that preprocesses the data responsibly, engineers meaningful features, trains a Linear Regression model, and delivers interpretable predictions with full documentation of societal implications.

9. Step-by-Step Procedure

1. Open Google Colab: go to colab.research.google.com and create a new notebook.
2. Mount Google Drive (optional) or upload dataset directly.
3. Install dependencies: !pip install scikit-learn pandas matplotlib seaborn
4. Import libraries: pandas, numpy, sklearn, matplotlib.pyplot, seaborn
5. Load dataset using pd.read_csv() and display shape and head(10).
6. EDA: Plot histograms, correlation heatmap, and check for missing values using df.isnull().sum().
7. Privacy audit: Drop or mask any PII columns. Document which columns were removed and why.
8. Handle missing values: use SimpleImputer with strategy='mean' for numerical columns.
9. Feature Engineering: Create BMI_Category (underweight/normal/overweight/obese) from BMI column.
10. Scale features: Apply StandardScaler to all numerical input features.
11. Split data: train_test_split with test_size=0.2 and random_state=42.
12. Train Linear Regression model: model.fit(X_train, y_train).
13. Predict on test set and compute MAE, MSE, RMSE, R^2 using sklearn.metrics.
14. Plot: (a) Actual vs Predicted scatter plot, (b) Residual plot, (c) Feature importance (coefficients).
15. Document ethical choices made during preprocessing in a Markdown cell.
16. Save model as diabetes_regression.pkl using joblib.dump().

10. Algorithm / Workflow

ALGORITHM Healthcare Regression Pipeline:

1. INPUT: Raw healthcare dataset (CSV file)
2. PROCESS:
   - a. Load → EDA → Identify missing values and outliers
   - b. Privacy Audit → Remove PII → Document removals
   - c. Impute missing → Encode categoricals → Scale numerics
   - d. Feature Engineering → Correlation analysis → Feature selection
   - e. Train-Test Split (80:20)
   - f. Fit Linear Regression → Predict → Evaluate
3. OUTPUT: MAE, RMSE, R^2 scores + Visualizations + Ethical Documentation

11. Expected Output

Correlation heatmap showing feature relationships

Training and test set sizes printed

MAE, RMSE, R^2 values printed in console

Actual vs Predicted scatter plot (points near diagonal = good fit)

Residual distribution histogram (should be approximately normal, centered at 0)

Feature coefficient bar chart (shows direction and magnitude of each feature's influence)

12. Result Analysis (Should be written in conclusion)

Answer the following based on your experiment:

- What is the R^2 score of your model and what does it mean?
- Which features have the highest positive and negative coefficients? What does this imply clinically?
- What was your MAE? Is this acceptable in a healthcare context?
- What ethical concerns arose during pre processing? How did you address them?
- How would you improve the model's performance? (suggest 2 techniques)

13. Ethical, Societal and Environmental Reflection

Using ML pipelines for healthcare prediction raises serious concerns about patient privacy, consent, and algorithmic bias.

Privacy: Patient data must be anonymized. Even after removing names, combinations of age, gender, and location can re-identify individuals (re-identification risk).

Bias: If training data over represents a demographic (e.g., urban, male, adult patients), the model performs poorly for underrepresented groups, potentially worsening health disparities.

Fairness: Healthcare ML must not discriminate based on protected attributes. Audit models using fairness metrics (demographic parity, equal opportunity).

Environmental Impact: Training large ML models on cloud GPUs has a carbon footprint. Use efficient models and consider green cloud providers.

Accountability: Clinicians must understand and be able to override ML recommendations. Explainability (using SHAP or LIME) is essential.

PRACTICAL NO. 2

FinTech Classification Logistic Regression, SVM, and XGBoost with ROC-AUC Analysis

Course Outcome	Learning Outcome	Performance Indicators
CO1: Create and evaluate end-to-end ML solutions with responsible data handling	LO 1.2: Evaluate and compare classification models on FinTech data using confusion matrix and ROC-AUC	PI-1.1.1, 1.4.1, 2.1.2, 2.4.1, 5.1.1, 5.2.2, 6.3.1, 6.3.2, 7.1.1, 7.2.1, 8.2.1

1. Aim / Objective

To evaluate and compare Logistic Regression, Support Vector Machine (SVM), and XGBoost classifiers on a financial fraud detection dataset, using confusion-matrix metrics and ROC-AUC curves to interpret model effectiveness.

2. Rationale

Financial fraud costs the global economy over $5 trillion annually. Banks and FinTech companies deploy ML classifiers to detect fraudulent transactions in real-time often with sub-millisecond latency requirements.

This practical builds on the ML Pipeline skills from Practical 1 and introduces multi-model comparison critical skill for data scientists who must justify model selection to business stakeholders.

Ethical significance: False negatives (missed fraud) cost the bank money; false positives (wrongly blocking genuine transactions) frustrate customers and may disproportionately impact low-income users.

3. SDG Mapping

SDG No.	SDG Title	Mapping Level	Justification
SDG 1	No Poverty	Medium	Fraud detection protects financially vulnerable individuals from scams and financial crime.
SDG 8	Decent Work and Economic Growth	High	Reliable FinTech systems support economic activity and trust in digital financial infrastructure.
SDG 16	Peace, Justice and Strong Institutions	High	Combating financial fraud strengthens rule of law and institutional integrity.

4. Prerequisite Knowledge

- Logistic Regression and sigmoid function
- SVM: hyperplane, support vectors, kernel trick (linear, RBF)
- Ensemble methods: boosting, XGBoost decision tree structure
- Confusion matrix: TP, TN, FP, FN; Precision, Recall, F1-Score
- ROC curve and AUC interpretation

- Class imbalance handling: SMOTE, class_weight parameter

5. Short Theory

5.1 Classification in FinTech

Classification assigns each transaction to a class: legitimate (0) or fraudulent (1). Financial datasets are typically highly imbalancedfraud may be < 0.1% of transactions.

5.2 Logistic Regression

Models $P(y=1|X)$ using the sigmoid function: $\sigma(z) = 1 / (1 + e^{-z})$. Interpretable, fast, and effective as a baseline. Assumes linearly separable classes.

5.3 Support Vector Machine (SVM)

Finds the optimal hyperplane that maximizes the margin between classes. The RBF kernel maps data to higher dimensions, enabling non-linear separation. C parameter controls regularization.

5.4 XGBoost

Gradient boosted decision trees. Each tree corrects errors of the previous. Key parameters: n_estimators (number of trees), max_depth, learning_rate, scale_pos_weight (for class imbalance).

5.5 Evaluation Metrics

Precision = TP/(TP+FP) | Recall = TP/(TP+FN) | F1 = 2×(P×R)/(P+R)

AUC-ROC: Area Under the ROC Curve. AUC = 1.0 → perfect; AUC = 0.5 → random classifier.

5.6 Class Imbalance

Use SMOTE (Synthetic Minority Oversampling Technique) or set class_weight='balanced' in sklearn to prevent models from ignoring the minority (fraud) class.

6. Software and Tools Required

- Python 3.10+
- Google Colab
- scikit-learn 1.4+
- XGBoost (pip install xgboost)
- imbalanced-learn (pip install imbalanced-learn)
- Pandas, NumPy, Matplotlib, Seaborn

7. Dataset Description

Dataset: Credit Card Fraud Detection Dataset (Kaggle)

Source: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

Size: 284,807 transactions, 31 features (V1–V28 are PCA-transformed, Time, Amount, Class)

Target Variable: Class (0 = Legitimate, 1 = Fraud)

Class Imbalance: 99.83% legitimate, 0.17% fraud highly imbalanced

8. Problem Statement

A FinTech company processing millions of digital transactions daily wants to build a real-time fraud detection classifier. Compare Logistic Regression, SVM, and XGBoost models on the credit card fraud

dataset. Handle class imbalance appropriately, evaluate all models using confusion matrix metrics and ROC-AUC curves, and recommend the best model with justification.

9. Step-by-Step Procedure

1. Open Google Colab and install required libraries: xgboost, imbalanced-learn.
2. Upload the creditcard.csv file to Colab or load via Kaggle API.
3. EDA: Check shape, class distribution (df['Class'].value_counts()), and summary statistics.
4. Visualize class imbalance using a bar/pie chart.
5. Preprocessing: Normalize 'Amount' and 'Time' using StandardScaler. Drop irrelevant columns.
6. Handle class imbalance: Apply SMOTE to training data only (never on test data).
7. Split data: 80% train, 20% test (stratified split to preserve class ratio).
8. Train Logistic Regression: LogisticRegression(class_weight='balanced', max_iter=1000).
9. Train SVM: SVC(kernel='rbf', class_weight='balanced', probability=True).
10. Train XGBoost: XGBClassifier(scale_pos_weight=ratio, use_label_encoder=False).
11. For each model: generate confusion matrix and classification report.
12. For each model: plot ROC curve and compute AUC score.
13. Plot all three ROC curves on a single figure for comparison.
14. Tabulate all metrics: Precision, Recall, F1, AUC for all three models.
15. Write a Model Selection Justification paragraph (which model and why).

10. Algorithm / Workflow

ALGORITHM FinTech Classification Pipeline:

1. INPUT: Raw credit card transaction data (CSV)
2. PROCESS:
   - a. EDA → Visualize class imbalance
   - b. Normalize Amount, Time → Train-test split (stratified)
   - c. Apply SMOTE on training set only
   - d. Train: LR → SVM → XGBoost
   - e. For each model: Predict → Confusion Matrix → Classification Report → ROC Curve
3. OUTPUT: Comparative metrics table + Multi-ROC plot + Model recommendation

11. Expected Output

Class distribution bar chart (showing severe imbalance)

After-SMOTE class distribution (balanced)

Confusion matrix for each of the 3 models

ROC curves for all 3 models on a single plot

Comparison table: Precision, Recall, F1, AUC for all models

12. Result Analysis (to be written as conclusion)

Answer the following based on your experiment:

- Which model achieved the highest AUC? What does AUC measure?
- Compare Recall scores across models. Why is high Recall critical in fraud detection?
- What happens if you do NOT apply SMOTE? Run the experiment and compare.

- Which model would you deploy in production and why? Consider both performance and speed.
- What are the trade-offs between Precision and Recall in this fraud detection context?

13. Ethical, Societal and Environmental Reflection

FinTech ML raises profound ethical concerns that go beyond technical accuracy.

False Positives in fraud detection: Blocking a legitimate transaction can leave a customer stranded especially dangerous for low-income users who have no backup payment method. This is a justice and equity issue.

Algorithmic Bias: If training data reflects historical discrimination (e.g., certain ethnicities flagged more often), the model will perpetuate this bias. Models must be audited for protected attribute fairness.

Explainability: Customers whose cards are blocked deserve a reason. The EU AI Act (2024) mandates explainability for high-risk AI systems including credit/fraud scoring.

Data Sovereignty: Transaction data is highly sensitive. Who stores it? In which country? Under what regulations? India's DPDP Act 2023 restricts cross-border data transfer.

Environmental Cost: XGBoost with thousands of trees has higher energy consumption than Logistic Regression. For real-time deployment at scale, energy efficiency is a legitimate design consideration.

PRACTICAL NO. 3

Neural Networks Perceptron and MLP on Linear and Non-Linear Datasets

Course Outcome	Learning Outcome	Performance Indicators
CO2: Design and train neural networks; optimize generalization and analyze real-world implications	LO 2.1: Design and train neural networks using Keras; analyze learning curves and decision boundaries	PI: 2.4.2, 3.2.1, 3.4.2, 4.3.1, 4.3.3, 5.1.1, 5.2.2

1. Aim / Objective

To build, train, and analyze Perceptron and Multi-Layer Perceptron (MLP) models using Keras on linearly separable (Iris, Moon) and non-linearly separable (XOR) datasets, and to visualize and compare decision boundaries and learning curves.

2. Rationale

Neural networks are the foundation of modern AIfrom voice assistants to medical imaging. Understanding their architecture from first principles (Perceptron → MLP) is essential for any AI practitioner.

This practical bridges the gap between theoretical neuroscience-inspired models and modern Keras implementations. Students move from the mathematical basis of a single neuron to a multi-layer network capable of solving non-linear problems.

The XOR problem famously proved that a single-layer perceptron cannot solve non-linear problemsthis limitation drove the development of multi-layer networks and the backpropagation algorithm.

3. SDG Mapping

SDG No.	SDG Title	Mapping Level	Justification
SDG 4	Quality Education	High	Neural network tools are reshaping education through adaptive learning platforms and intelligent tutoring systems.
SDG 9	Industry, Innovation and Infrastructure	High	MLPs power industrial automation, predictive maintenance, and smart infrastructure systems.

4. Prerequisite Knowledge

- Biological neuron analogy and McCulloch-Pitts model
- Activation functions: Step, Sigmoid, ReLU, Tanh
- Gradient descent and the learning rate concept
- Basic Keras Sequential API usage
- Python: NumPy array operations, Matplotlib plotting

5. Short Theory

5.1 The Perceptron

A Perceptron is the simplest neural unit. It computes a weighted sum of inputs, adds a bias, and applies an activation function. For input vector X and weights W:

$$\text{output} = f(W \cdot X + b)$$

A single Perceptron can only solve linearly separable problems (AND, OR) but FAILS on XOR.

5.2 Multi-Layer Perceptron (MLP)

MLP adds one or more hidden layers between input and output. Hidden layers learn intermediate feature representations. With enough neurons and non-linear activations, MLP is a universal function approximator (Universal Approximation Theorem).

5.3 Activation Functions

- ReLU: $f(x) = \max(0, x)$avoids vanishing gradient; default for hidden layers.
- Sigmoid: $f(x) = 1/(1+e^{-x})$squashes to [0,1]; good for binary output.
- Softmax: converts logits to probability distribution; used for multi-class output.

5.4 Backpropagation

Error propagates backward through the network using the chain rule of calculus. Weights are updated: $W\_new = W\_old - \eta \times \partial L/\partial W$, where η is the learning rate and L is the loss function.

5.5 Decision Boundaries

A linear model produces a straight-line boundary. MLP produces curved boundaries enabling it to correctly classify XOR and other non-linear patterns.

6. Software and Tools Required

- Python 3.10+
- Google Colab
- TensorFlow 2.x / Keras
- scikit-learn (datasets, metrics)
- Matplotlib, NumPy

7. Dataset Description

Dataset 1: Iris Dataset150 samples, 4 features, 3 classes (linearly separable in reduced form)

Dataset 2: Moons Data set sklearn.datasets.make_moons()1000 samples, 2 classes, non-linear

Dataset 3: XOR Data set manually constructed4 points, illustrates non-linear separation

All datasets are generated or loaded via scikit-learnno download required.

8. Problem Statement

Demonstrate that a single-layer Perceptron can solve linearly separable problems (Iris/Moons) but fails on non-linear problems (XOR). Implement an MLP with one hidden layer and show that it successfully learns the XOR function. Visualize and compare decision boundaries and learning curves for both architectures.

9. Step-by-Step Procedure

1. Open Google Colab and import TensorFlow/Keras, sklearn datasets, NumPy, Matplotlib.
2. Generate XOR data: 4 points (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→0.
3. Load Iris dataset (binary: classes 0 and 1 only) and Moon dataset.
4. Build Perceptron model: Dense(1, activation='sigmoid') on Iriscompile, train, evaluate.
5. Build Perceptron model on XOR observe that it fails to converge below 50% accuracy.
6. Build MLP on XOR: Input(2) → Dense(4, ReLU) → Dense(1, Sigmoid). Train and verify 100% accuracy.
7. Build MLP on Moons dataset: Input(2) → Dense(8, ReLU) → Dense(4, ReLU) → Dense(1, Sigmoid).
8. For each model, plot: (a) Learning curve (accuracy/loss vs epoch), (b) Decision boundary.
9. Use mlxtend or manual meshgrid to plot decision boundaries.
10. Compare: Perceptron vs MLP on the same datasettabulate accuracy.
11. Build MLP on full Iris (3-class): Dense(16, ReLU) → Dense(3, Softmax). Evaluate.

10. Algorithm / Workflow

ALGORITHM Perceptron and MLP:

1. Perceptron on Linearly Separable Data:
   - Input → $\Sigma(w_i x_i + b)$ → Sigmoid → Binary output → Backprop → Converges
2. Perceptron on XOR:
   - Input → $\Sigma(w_i x_i + b)$ → Sigmoid → Binary output → Backprop → FAILS TO CONVERGE
3. MLP on XOR:
   - Input(2) → Hidden(4, ReLU) → Output(1, Sigmoid) → Backprop → Converges to 100%

KEY INSIGHT: Hidden layer enables non-linear decision boundary solves XOR.

11. Expected Output

Decision boundary plot: Perceptron on XOR linear boundary fails to separate classes

Decision boundary plot: MLP on XOR curved boundary correctly separates all 4 points

Learning curves: Perceptron accuracy on XOR stays ~50%; MLP converges to 100%

MLP on Moons: curved decision boundary separating two interleaved half-moons

MLP on Iris: Training and validation accuracy curves showing convergence > 95%

12. Result Analysis

Answer the following based on your experiment:

- What accuracy did the Perceptron achieve on XOR? Why did it fail?
- What accuracy did the MLP achieve on XOR? What architectural change enabled this?
- Compare the decision boundaries of Perceptron and MLP on the same plot. Describe the difference.
- Which activation function did you use in the hidden layer? Why?
- What does the learning curve tell you about the training process?

13. Ethical, Societal and Environmental Reflection

Neural networks are increasingly used in high-stakes decisions loan approvals, parole decisions, medical diagnoses raising serious ethical questions.

Bias Amplification: If training data contains human bias (e.g., historical loan denial patterns), the MLP will learn and amplify that bias at scale.

Explainability Problem: Unlike Decision Trees, MLPs are black boxes. A patient denied a diagnosis or a loan denied by an MLP deserves an explanation but neural networks do not naturally provide one.

Energy Consumption: Training deep networks requires significant GPU compute. GPT-4 training reportedly used ~1 GWh of electricity. Even small lab experiments contribute to cumulative carbon footprint. Use Colab's free tier efficiently.

Over-reliance: Organizations may over-rely on neural network outputs without human oversight. Establish human-in-the-loop (HITL) validation for critical applications.

PRACTICAL NO. 4

Deep Neural Networks MNIST and CIFAR-10 with Activation Functions and Optimizers

Course Outcome	Learning Outcome	Performance Indicators
CO2: Design and train neural networks; optimize generalization and analyze real-world implications	LO 2.2: Design, implement, optimize, and evaluate DNNs for image classification tasks	PI-2.4.2, 2.4.4, 3.2.2, 3.4.1, 4.2.1, 4.3.2, 6.3.1, 7.1.1, 8.2.1, 9.1.2, 9.2.2

1. Aim / Objective

To design and train Deep Neural Networks (DNNs) using multiple activation functions and optimizers on MNIST and CIFAR-10 datasets; experiment with learning rates, evaluate performance, and critically analyze technical, societal, ethical, and environmental implications.

2. Rationale

MNIST is the 'Hello World' of deep learning60,000 handwritten digit images used in bank cheque processing and postal sorting. CIFAR-10 contains 60,000 real-world images spanning 10 categories used in medical imaging, surveillance, and autonomous vehicle perception.

This practical moves from simple MLPs (Practical 3) to deeper architectures with multiple hidden layers, exploring how depth and different optimizers (SGD, Adam, RMS Prop) affect convergence speed and accuracy.

A DNN trained on MNIST can automate cheque reading, reducing processing time from minutes to milliseconds with direct economic and societal impact.

3. SDG Mapping

SDG No.	SDG Title	Mapping Level	Justification
SDG 9	Industry, Innovation and Infrastructure	High	DNNs power intelligent industrial automation, autonomous vehicles, and smart city infrastructure.
SDG 3	Good Health and Well-Being	High	Medical imaging DNNs (X-ray, MRI) reduce diagnostic delays and improve healthcare access.
SDG 11	Sustainable Cities and Communities	Medium	Postal sorting automation reduces urban logistics costs and improves service delivery.

4. Prerequisite Knowledge

- MLP architecture and Keras Sequential API (Practical 3)
- Image data representation: pixels, channels, flatten operation
- Categorical cross-entropy loss for multi-class classification
- One-hot encoding of labels

- Learning rate and its effect on gradient descent convergence

5. Short Theory

5.1 From MLP to DNN

A Deep Neural Network is simply an MLP with many hidden layers (typically 3+). Depth allows learning hierarchical features early layers detect edges, middle layers detect shapes, deep layers detect objects.

5.2 Activation Functions Comparison

- ReLU: Most popular. Fast computation. Avoids vanishing gradient. Risk: dying ReLU (neurons stuck at 0).
- Sigmoid: Used in output layer for binary. Saturates at extremes causes vanishing gradient in deep networks.
- Softmax: Multi-class output. Converts logit vector to probability distribution summing to 1.

5.3 Optimizers

- SGD (Stochastic Gradient Descent): Updates weights using one or mini-batch of samples. Slow but stable. With momentum: faster convergence.
- Adam (Adaptive Moment Estimation): Combines momentum + RMSProp. Adaptive learning rate per parameter. Most popular default optimizer.
- RMSProp: Divides learning rate by exponentially decaying average of squared gradients. Good for recurrent networks.

5.4 MNIST Dataset

70,000 grayscale images (28×28 pixels) of handwritten digits 0–9. Each image is flattened to 784 features. The DNN learns a 784→512→256→10 mapping.

5.5 CIFAR-10 Dataset

60,000 color images (32×32×3 = 3072 features) in 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck. Much harder than MNIST due to color and texture variation.

6. Software and Tools Required

- Python 3.10+
- Google Colab (GPU runtime recommended for CIFAR-10)
- TensorFlow 2.x / Keras
- Matplotlib, NumPy
- scikit-learn

7. Dataset Description

Dataset 1: MNISTBuilt into Keras (keras.datasets.mnist)

60,000 training + 10,000 test images | 28×28 grayscale | 10 classes (digits 0–9)

Dataset 2: CIFAR-10Built into Keras (keras.datasets.cifar10)

50,000 training + 10,000 test images | 32×32×3 RGB | 10 object classes

No download required both datasets load directly via Keras.

8. Problem Statement

A bank wants to automate cheque processing using handwritten digit recognition (MNIST). A smart city project needs object classification for surveillance camera feeds (CIFAR-10). Design and train DNNs for both tasks, experiment with activation functions and optimizers, and analyze how architecture choices affect accuracy and training time.

9. Step-by-Step Procedure

1. Open Colab and enable GPU: Runtime → Change Runtime Type → GPU.
2. Load MNIST dataset: keras.datasets.mnist.load_data(). Normalize pixel values to [0,1].
3. Flatten images: reshape to (60000, 784).
4. Encode labels: keras.utils.to_categorical(y_train, 10).
5. Build DNN: Input(784) → Dense(512, ReLU) → Dense(256, ReLU) → Dense(128, ReLU) → Dense(10, Softmax).
6. Compile with Adam optimizer, categorical_crossentropy loss, accuracy metric.
7. Train for 20 epochs with validation_split=0.1. Record history.
8. Experiment 1: Replace ReLU with Sigmoid in all hidden layers. Re-train. Compare.
9. Experiment 2: Replace Adam with SGD, then RMSProp. Re-train each. Compare.
10. Load CIFAR-10: keras.datasets.cifar10.load_data(). Normalize to [0,1]. Flatten to (50000, 3072).
11. Build DNN for CIFAR-10 (same or deeper architecture). Train with Adam.
12. Plot accuracy/loss curves for all experiments on the same figure.
13. Tabulate: Model | Optimizer | Activation | Val Accuracy | Epochs to converge | Training time.

10. Algorithm / Workflow

ALGORITHMDNN Training:

1. Load data → Normalize → Flatten → One-hot encode labels
2. Build DNN: Input → [Dense(n, activation)] × L layers → Dense(C, Softmax)
3. Compile: optimizer (SGD / Adam / RMSProp) + loss (categorical_crossentropy)
4. Train: model.fit(X_train, y_train, epochs=E, validation_split=0.1)
5. Evaluate: model.evaluate(X_test, y_test)
6. Visualize: accuracy curve, loss curve, sample predictions
7. Repeat for different activations and optimizers. Compare results.

11. Expected Output

MNIST sample images grid (2×5) showing handwritten digits

Validation accuracy curves for all 4 experiments on one plot (ReLU+Adam should be highest)

Validation loss curves for all 4 experiments

CIFAR-10 sample images grid (2×5) showing objects

CIFAR-10 training/validation accuracy and loss curves

Expected MNIST test accuracy: ReLU+Adam ~98%, Sigmoid+Adam ~97%, SGD ~96-97%, RMSProp ~98%

Expected CIFAR-10 DNN accuracy: ~50-55% (much lower; motivates CNNs in Practical 6)

12. Result Analysis

Answer the following based on your experiment:

- Which activation+optimizer combination gave the best MNIST accuracy? Why do you think this is?
- How does the validation accuracy differ from training accuracy? What does a large gap indicate?
- What CIFAR-10 accuracy did your DNN achieve? Why is it lower than MNIST accuracy?
- Compare convergence speed: which optimizer reached 95% accuracy first?
- What does the learning curve tell you about choosing the number of epochs?

13. Ethical, Societal and Environmental Reflection

Deep Neural Networks embedded in surveillance cameras, ATMs, and postal systems raise significant societal concerns.

Surveillance and Privacy: CIFAR-10 object classifiers deployed in surveillance cameras can identify people's locations, movements, and associations without their consent enabling mass surveillance by governments or corporations.

Algorithmic Bias in Handwriting Recognition: MNIST-trained models may underperform on handwriting styles from non-Western populations, causing inequitable service in international banking.

Environmental Cost of Deep Learning: Training a DNN on CIFAR-10 on a GPU for 20 epochs produces measurable CO_2. ImageNet-scale training emits as much CO_2 as five round-trip flights from Mumbai to New York. Use model compression and efficient architectures.

Autonomous Vehicle Safety: A misclassification in a CIFAR-10-like system embedded in an autonomous vehicle (mistaking a person for a truck) can be fatal. Rigorous validation and fail-safe mechanisms are mandatory.

Accountability Gap: When an ATM refuses a cheque due to a DNN misread, who is accountable? Banks must maintain human review pathways.